

Nucleus - ROOT AI Native

全栈岗位笔试说明

候选人：李龙勇
项目名称：Nucleus
项目定位：可恢复、可审计的多 Agent 网页应用生成平台
最终功能代码基线：b14e81e00d23865640fe318c9ef81abcd82bcf24

0. 笔试结果回收信息

可直接将下面三项复制到原笔试文档的“4. 笔试结果回收通道”：

- **笔试说明文档 (PDF)**： <https://github.com/lilongyong333/nucleus-ai-builder/blob/agent/metagpt-quality-gate/output/pdf/Nucleus-ROOT-Fullstack-Written-Test-Li-Longyong.pdf>
- **已部署的可测试链接**： <https://www.llynb.cc>
- **GitHub 源码**： <https://github.com/lilongyong333/nucleus-ai-builder/tree/agent/metagpt-quality-gate>

辅助验收链接：

- 可在线阅读的 Markdown 说明： <https://github.com/lilongyong333/nucleus-ai-builder/blob/agent/metagpt-quality-gate/docs/SUBMISSION.md>
- Sites 备用地址： <https://nucleus-ai-builder-root.dreamy-joy-4746.chatgpt.site>
- 真实生成的贪吃蛇成品： <https://www.llynb.cc/p/responsive-snake-game-d1bf0e>
- 对应工作台与 Agent 审计： <https://www.llynb.cc/w/d1bf0eb6-4b74-48d0-984e-ebaa773cbb3c>
- 最终 GitHub PR 与 CI： <https://github.com/lilongyong333/nucleus-ai-builder/pull/2>

说明：在线域名运行的是上一版稳定部署；最终 GitHub 分支额外包含商业控制面基础代码。两者分开标注，避免把“已经实现但尚未配置外部凭据”的能力描述成“已经在线开通”。

1. 项目概述

Nucleus 是一个面向非技术用户的 AI 应用生成器。用户输入一句自然语言需求后，Iris、Bob、Alex、Ray 四个 Agent 会依次完成需求分析、架构设计、代码生成和质量审查，最终交付一个可点击、可继续修改、可保存版本并可公开分享的网页应用。

我对题目的理解不是“做一个会返回代码的聊天框”，而是完成一条可验证的产品闭环：

- 输入需求
- > 看到真实 Agent 协作过程
 - > 得到可交互应用
 - > 保存数据与版本
 - > 继续对话修改
 - > 质量检查与失败恢复
 - > 发布公开链接或下载源码

这个闭环直接对应题目要求的真实交互、数据持久化、核心使用流程、至少一个衍生能力、在线访问链接和实际可用功能。

2. 题目要求完成度

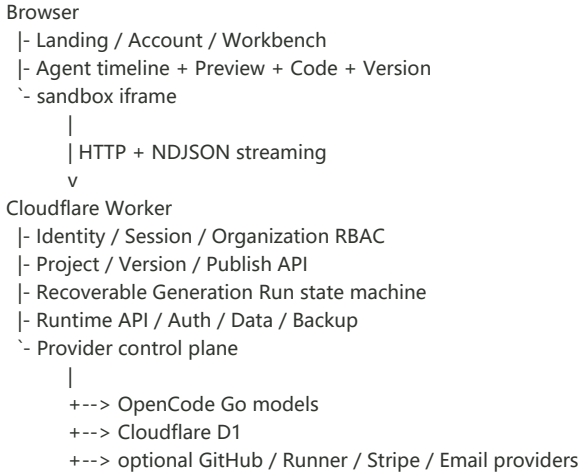
原题要求	Nucleus 的实现	状态
类似 Atoms 的 Agent 驱动应用生成	Iris/Bob/Alex/Ray 按真实阶段生成结构化工件和三文件应用	已完成
生成结果可视化展示	sandbox iframe 实时预览，支持桌面/手机视口和源码查看	已完成
真实交互，而非静态页面	生成应用可以点击、输入、增删改数据、键盘操作和运行游戏	已完成
数据持久化	D1 保存项目、消息、Run、Agent 事件、工件、版本和运行数据	已完成
基本使用流程	游客直接体验；支持账号登录、生成、修改、版本、发布和下载	已完成
至少一个衍生能力	断线恢复、Ray 自动修复、Race Mode、可视化局部修改等	已完成，多项
可测试在线链接	www.llynb.cc 及备用 Sites 地址	已完成
GitHub 源码	公开仓库、最终分支、PR 和 CI 记录	已完成

3. 框架与系统设计

3.1 技术选型

层级	方案	主要职责
Web 前端	React 19、Vinxent App Router、TypeScript、原生 CSS	首页、工作台、账号项目库、预览、代码和版本交互
API/运行时	Cloudflare Worker	项目 API、生成状态机、身份、发布、应用运行时 API
数据层	Cloudflare D1、Drizzle Schema/Migration	项目、对话、版本、Agent 审计、应用数据与控制面状态
AI 模型层	OpenAI-compatible OpenCode Go API	Iris/Bob/Alex/Ray 的真实规划、生成、评审和修复
生成物运行	sandbox iframe + Runtime Bridge	隔离运行 HTML/CS S/JavaScript，回传 ready/error/consol e 证据
工程质量	Vitest、Playwright、ESLint、TypeScript、GitHub Actions	单元、固定 Eval、真实浏览器、构建和远端 CI 验收

3.2 总体架构



3.3 多 Agent 生成链路

1. **Iris - 需求规划**：将一句话需求转换为功能、验收标准、风险和测试计划；
2. **Bob - 系统架构**：设计页面结构、状态模型、交互流、三文件职责和 Runtime Blueprint；
3. **Alex - 实际开发**：分阶段生成 `index.html`、`styles.css`、`script.js`，每个工件完成后立即落库；
4. **Ray - 质量审查**：执行语法、安全、交互、响应式和应用类型专项检查，失败时最多进行有限次数的定向修复；
5. **Finalize**：只有质量门通过才创建 Version，随后在隔离 iframe 中运行并允许发布。

四个 Agent 对应数据库中真实的 Stage、Artifact、ModelAttempt 和 AgentEvent，不是前端延时动画模拟出来的“多 Agent”。

4. Demo 已实现功能

4.1 用户可直接体验的核心功能

- 游客无需注册即可创建项目；登录后项目、版本和对话可以跨设备保存；
- 自然语言生成 HTML/CSS/JavaScript 三文件应用；
- 真实流式 Agent 进度、阶段状态、模型尝试、耗时和 Token 审计；
- 可交互实时预览、源码查看、桌面/移动视口切换；
- 基于当前版本继续对话修改；生成期间可将下一条消息加入队列；
- 项目和完整版本快照持久化，支持历史版本恢复；
- 固定版本公开发布，后续草稿不会悄悄改变已分享的成品；
- ZIP 源码下载；
- 启动错误、Console error 和 Promise rejection 回传工作台；
- 失败状态、v0 空状态和 Provider 未配置状态均明确展示，不用固定 Demo 冒充新结果。

4.2 工程可靠性能力

- Run 和 Stage 两级状态机，每个阶段都有 D1 检查点；
- 浏览器流断开后从服务端当前阶段恢复，不要求整轮重跑；
- 项目租约、阶段租约和 generation ID 防止双击、多个标签或迟到响应覆盖新版本；
- 主备模型路由、阶段调用/Token/时间预算和取消传播；

- 模型输出协议门：文件边界、Markdown 残留、HTML/CSS 完整性和 Acorn JavaScript 语法；
- Ray 确定性质量门、模型审查和有限自动修复；
- 运行证据、备份、恢复前检查点和审计记录。

5. 创新性与差异化设计

5.1 “诚实生成”，而不是套用预制 Demo

新项目从 v0 开始。只有模型真正生成完整工件并通过 Ray 才创建 v1；超时、截断、协议污染或质量失败都会保留明确失败状态。这个设计解决了 AI Builder 最容易被质疑的问题：界面显示“生成成功”，实际却只是复用了旧模板。

5.2 可恢复、可审计的多 Agent 工作流

传统 Demo 往往把四个角色写成几段 Loading 文案。Nucleus 将角色落实为可持久化状态机：每次模型调用、首字时间、输出长度、Token、错误、Artifact 和最终 Version 都能追踪。网络中断后可以继续未完成阶段，也能解释“为什么失败、在哪里失败”。

5.3 生成物从静态网页升级为应用运行时

每个正式版本都有 AppManifest，声明 API、集合 Schema、Auth、依赖和验收要求。生成应用可通过 `window.nucleus` Runtime SDK 使用数据 CRUD、会话、日志和备份，而不只是刷新后数据消失的静态 HTML。

5.4 Race Mode 与证据驱动修复

Race Mode 可以让多个代码模型并行生成候选，通过确定性指标评分并记录选择证据。预览中的 Console、Runtime、DOM 和外部浏览器证据可以回灌 Ray，用实际错误驱动修复，而不是只让模型“再试一次”。

5.5 面向继续产品化的控制面

最终代码还提供了组织 RBAC、发布审批、用量事件、Agent Git 分支、外部 Playwright/容器 Runner 协议、可选物理 D1 Provisioner、GitHub App、Stripe、邮件、维护任务和 SLO 评估基础。所有外部 Provider 缺少凭据时都会返回 `configuration-required`，不会把尚未开通的云服务描述为成功。

6. 关键工程取舍

决策	当前选择	原因与代价
默认生成格式	无构建步骤的三文件网页应用	预览快、成本低、演示稳定；复杂 npm 项目交给外部 Runner
生成代码执行	浏览器 sandbox iframe	Worker 不执行任意 shell/Docker，降低沙箱逃逸风险
长任务模型	分阶段短请求 + D1 检查点	比单个超长请求更容易恢复和审计，但状态机更复杂
版本模型	全量快照	小型生成应用下恢复最可靠，空间开销可接受
数据库隔离	默认项目级逻辑隔离，可选独立物理 D1	笔试环境开箱即用，同时保留商业化扩展路径

决策	当前选择	原因与代价
身份	游客 HttpOnly 工作区 + ChatGPT 登录	评审打开即用，登录后可跨设备；不自行保存密码
失败处理	明确失败，不自动展示旧 Demo	牺牲“每次看起来都成功”，换取结果真实性

7. 当前未完成或未上线的部分

能力	当前真实状态
最终 GitHub 商业控制面代码部署到 www.llynb.cc	尚未部署；在线地址保持上一版稳定版本
托管的任意 Node/Python/Java 容器集群	已有 Provider 协议、配额和回调；没有附带真实集群
每项目物理 D1	Provisioner 和 Migration 已实现；需要 Cloudflare API Token 后真实开通
GitHub App	OAuth/JWT/Installation Token 代码已实现；需要注册 App 和配置私钥
Stripe 和邮件	真实 API 集成骨架已实现；没有配置商户、价格、域名和生产密钥
大规模生成成功率	已有固定 Eval，但没有足够大的多类型真实生成样本，不能宣称统计成功率
多用户高负载与安全攻防	有只读负载脚本和安全契约测试；尚未完成生产规模压测和专业渗透测试

8. 如果继续投入：扩展优先级

P0 - 先保证最终版本可交付

1. 将最终分支部署到 staging，执行 D1 0008 Migration；
2. 跑登录、生成、恢复、发布、公开页和移动端完整烟雾测试；
3. 验证 Cloudflare 日志、告警和一键回滚后再切换生产域名。

P1 - 打通真实外部 Provider

1. 注册 GitHub App 并完成安装、分支、PR 的真实端到端验收；
2. 接入受限容器 Runner 和云端 Playwright，按截图/Console 自动回灌 Ray；
3. 为物理 D1 Provisioner 配置最小权限 Token，验证创建、迁移、备份和删除保留期；
4. 配置 Stripe Test Mode 与邮件测试域名，验证 Checkout、Webhook 幂等、邀请和通知。

P2 - 用数据提升生成成功率

1. 建立覆盖看板、表单、CRUD、游戏、数据可视化和移动端的固定 E2E Eval 集；
2. 记录首次成功率、修复后成功率、P95 时延、Token 成本和回归趋势；
3. 根据失败类型优化 Prompt、模型路由、Runtime SDK 和质量门，而不是继续堆功能。

P3 - 商业化和平台治理

1. 多租户配额、账单、超额计费和组织审计；
2. 备份演练、跨区域灾备、SLO、值班和故障复盘；
3. 依赖投毒、恶意 Prompt、沙箱逃逸和高并发压力测试。

9. 验证结果

最终功能提交 `b14e81e` 的实际验收结果：

- Vitest 单元/集成测试： **201/201**；
- 固定产品与安全 Eval： **155/155**；
- Chromium Playwright 真实浏览器 E2E： **7/7**；
- TypeScript： 通过；
- ESLint： 通过；
- Drizzle Schema/Migration 一致性： 通过；
- Vinext production build： 通过；
- GitHub Actions Linux CI： 通过；
- Git diff、常见密钥格式和原始笔记 PDF 提交检查： 通过。

固定 Eval 验证的是确定性产品与安全契约，不等价于 “任意需求生成成功率 100%” 。

10. AI 工具使用说明

- **Codex**： 用于需求拆解、代码实现、重构、测试、排障、GitHub 交付和文档整理；
- **OpenCode Go 模型 API**： 作为 Nucleus 产品内部 Iris/Bob/Alex/Ray 的真实模型来源；
- **人工判断**： 决定产品范围、失败是否可以接受、功能优先级、工程边界和最终交付标准。

项目保留了失败 Run、模型尝试、生产故障和逐轮修复证据。完整教学和复现材料位于 `docs/learning/`，逐轮验收记录位于 `docs/PROGRESS.md`。

11. 建议演示顺序（3 分钟）

1. 打开首页，说明游客可直接使用，登录后可跨设备保存；
2. 打开贪吃蛇工作台，展示 Iris/Bob/Alex/Ray 的阶段、Artifact、模型尝试和 Ray 质量结果；
3. 打开公开成品，实际操作开始、暂停、方向控制和重新开始；
4. 回到工作台展示 Preview/Code、对话修改、版本恢复、发布和 ZIP 下载；
5. 最后说明断线恢复、v0 真实性和 `configuration-required`，体现工程可靠性与边界意识。

一句话总结：**Nucleus 不只是把模型输出显示在网页上，而是把应用生成做成了一条可运行、可持久化、可恢复、可审计、可继续扩展的工程闭环。**